MeshOn: Intersection-Free Mesh-to-Mesh Composition

Supplementary Material

A. Technical Details

In this section, we will delineate some of the technical details and derivations we have left out in the method section of our main paper.

A.1. Distance Measures

Recall that at the core of both our proximity loss $\mathcal{L}_p$ and the *Incremental Potential Contact* barrier energy, $\mathcal{L}_{IPC}$ is the calculation of a vertex-to-surface distance measure between the two meshes.

Let $\mathcal{M}_o$ denote our object mesh and $\mathcal{M}_b$ our base mesh. The objective of this distance metric is to give an efficient measure of distance between the user-defined localized region on the base mesh, denoted by $\mathcal{M}_b^*$, and $\mathcal{M}_o$, and vice-versa. The minimum distance between the surface of any two triangular meshes is the minimum of the minimum pairwise edge-to-edge and triangle-to-vertex distance. In practice, we observe that using only the triangle-to-vertex distance gives us a practically "good-enough" measure of the surface-wise distance without causing too much memory overhead. Intuitively, for measuring the vertex-to-triangle distance from $\mathcal{M}_o$ to $\mathcal{M}_b^*$, we calculate the distance from all vertices on $\mathcal{M}_o$, v_i, to the triangles on $\mathcal{M}_b^*$, $\mathcal{F}_b$. In order to reduce memory overhead in gradient back-propagation, we follow [2] and create a masked distance function $\hat{d}$ by thresholding the minimum triangle-to-vertex distance from each vertex v_i on the object mesh to the triangles on the localized surface of the base mesh, $\mathcal{M}_b^*$:

$$\hat{d}(v_i, \mathcal{M}_b^*) = \begin{cases} d(v_i, \mathcal{M}_b^*) & \text{if } \min_{\mathcal{F}_b \in \mathcal{M}_b^*} d(v_i, \mathcal{F}_b) < \ell, \\ 0 & \text{otherwise,} \end{cases}$$

where $\ell = 0.5$. We discarding any triangle-to-point distance pairs that are far enough so their gradients have no meaningful influence over the alignment process.

Proximity Loss. Using this measure, we approximate the proximity between the two mesh surfaces by calculating the vertex-to-face measure, $\hat{d}$, for both $\mathcal{M}_b^*$ and $\mathcal{M}_o$.

Our proximity loss is naturally defined as

$$\mathcal{L}_p(\theta) = \sum_{v_i \in \mathcal{M}_o} \hat{d}(R(\theta)v_i, \mathcal{M}_b^*)^2 + \sum_{v_j \in \mathcal{M}_b^*} \hat{d}(v_j, R(\theta)\mathcal{M}_o)^2 , \tag{1}$$

During optimization, we update the pairwise triangle-to-vertex distance measure, $\min_{\mathcal{F}_b \in \mathcal{M}_b^*}$, thereby iteratively updating which triangle-vertex pairs are to be brought closer through the optimization process. Our key insight is that by iteratively updating these pairs, the object mesh $\mathcal{M}_o$ eventually moves in a direction that not only minimizes the shortest distance between the two surfaces, but also induces as many triangle-vertex pairs to be in close contact to each other, eventually finding a good "fit" between the two meshes.

Barrier Loss. We also reuse the contact measure $d(v_i, \mathcal{F}_b)$ to derive a loss term that prevents physical intersection between the two surfaces. We define a contact set based on a smaller threshold:

$$\mathcal{C} = \{i \mid \min_{\mathcal{F}_b \in \mathcal{M}_b^*} d(v_i, \mathcal{F}_b) < \ell\},$$

where h is a much smaller value than that of the proximity loss ($\ell = 0.01$). This is because in order to prevent contact, we only need to account for the triangle-to-vertex pairs that are close enough such to cause surface intersections.

To differentiably penalize against proximity for these near-contact vertices, we adopt the clamped log-barrier energy from [2]. Given the pre-filtered distances d_i corresponding to vertices in $\mathcal{C}$, the barrier energy is defined as:

$$b_h(d_i) = -(d_i - h)^2 \ln\left(\frac{d_i}{h}\right), \qquad \text{for } d_i < h.$$

This energy smoothly penalizes points as they approach the base surface, with a second-order continuous (C^2) form at the boundary $d_i = h$, and naturally vanishes as $d_i \to h$ or beyond. As a result, the total contact loss is expressed as:

$$\mathcal{L}_{\text{IPC}}(\theta) = \sum_{v_i \in \mathcal{M}_o} b_h\left(d\left(R(\theta)v_i, \mathcal{M}_b\right)\right) . \tag{2}$$

We additionally note that this distance measure is implemented as a batched tensorized operation in Pytorch, saving us from for-looping through the entire list of vertices and faces.

A.2. Vector-Optimized Bounding Volume Hierarchy

Unfortunately, calculating $d(v_i, \mathcal{F}_b)$ requires $\mathcal{O}(|\mathcal{V}_o||\mathcal{F}_b|)$ computational complexity, where $\mathcal{V}_o$ is the number of vertices and $\mathcal{F}_b$ the number of faces involved. In other words, the computation could become infeasibly expensive as mesh resolution scales.

To overcome this challenge while adequately taking advantage of our method's tensorized nature, we employ a

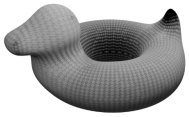
	340 Faces	5062 Faces	10688 Faces	256512 Faces	1026048 Faces
Min Leaf Size					
Full Batch	Runtime: 0.0077sec Memory: 91.42MB	Runtime: 0.0277sec Memory: 1328.35MB	Runtime: 0.1541sec Memory: 2801.10MB	Out-of-memory	Out-of-memory
1000 Triangles	Runtime: 0.0511sec Memory: 13.41MB	Runtime: 0.0602sec Memory: 179.74MB	Runtime: 0.0995sec Memory: 382.55MB	Runtime: 2.5897sec Memory: 8876.17MB	Runtime: 10.1058sec Memory: 35500.60MB
100 Triangles	Runtime: 0.2457sec Memory: 3.42MB	Runtime: 0.2563sec Memory: 24.13MB	Runtime: 0.3613sec Memory: 49.42MB	Runtime: 1.3816sec Memory: 1127.50MB	Runtime: 5.1874sec Memory: 4500.07MB
1 Triangle	Runtime: 1.8164sec Memory: 3.42MB	Runtime: 1.8568sec Memory: 4.35MB	Runtime: 2.6464sec Memory: 7.41MB	Runtime: 3.5156sec Memory: 143.39MB	Runtime: 3.3279sec Memory: 564.20MB

Figure 1. We show a comparison of runtime and GPU memory consumption for calculating a single distance measure d using our GPU-optimized BVH structure. We run the calculation with varying resolutions of the base mesh (visualized on top of each mesh), without using any user defined masks. We use a target mesh with a constant size of 8646 vertices and 17288 faces.

Vector-Optimized Bounding Volume Hierarchy (BVH) structure that efficiently parses through a BVH tree before it reaches an optimal depth and falls back to a tensorized batch query of all triangle instances within its child nodes.

Building the Tree To build the vector-optimized BVH over a face subset $\mathcal{F}$, our goal is to recursively calculate and store the minimum and maximum bounding box, $\mathbf{b}_{\min} = \min_{f\in[s:e)} \mathbf{m}_f$ and $\mathbf{b}_{\max} = \max_{f\in[s:e)} \mathbf{M}_f$ for each node, where $[s:e)$ is range of triangle indices in the node (and its children). Beginning with the top-most parent node with $[s:e) = [0, num(F))$ and $(\mathbf{b}_{\min}, \mathbf{b}_{\max})$ that encompass all triangles $\mathcal{F}$, we select the longest axis of this bounding box and sort the child nodes according to the center of their bounding boxes along the selected axis, c_i. We then split each node at the median of this sorted list, a process that we recursively repeat until the number of child node reaches a user-specified `leaf_size`. The key contribution of our vector-optimized BVH is that we stop building this tree when $e - s \leq$ `leaf_size`, where `leaf_size` is a variable set either by the user, or by some memory specification. To optimally take advantage of the vectorized nature of our distance calculation while preventing memory overhead from massive matrix multiplication, it is optimal to stop the tree traversal at a gpu-specific depth, and do a batched calculation of $d(n^{(i)}_{\text{vert}}, T)$ over triangles in the range $f \in [s:e)$. For simplicity, we set `leaf_size` $= 1000$ for all experiments run in this paper.

Hybrid Best First Traversal The main challenge of using a BVH $\mathcal{M}_o$ is that the mesh is continuously transformed each iteration. For rigid transformations (rotation, translation, and scale), the BVH bounding boxes can be efficiently updated by transforming their centers and recalculating new, slightly looser half-extents.

For non-rigid transformations, we recalculate BVH structure every $cycle = 20$ iterations, assuming the boxes stay relatively axis-aligned and the vertex positions relatively stable. This allows us to reuse the same BVH structure for small number of cycles with minimal sacrifice in accuracy.

A.3. VLM

Now we detail how we use Vision Language Models (VLM) to obtain a coarse initialization for Step 1.

Given randomly initialized meshes $\mathcal{M}_o$ and $\mathcal{M}_b$, we aim for a good initialization of $\mathcal{M}_o$ by optimizing its orientation using a Vision-to-Language Model (VLM), while determining scale and translation heuristically. We utilize a multi-agent VLM pipeline responsible for defining semantic alignment criteria and classifying rendered views to output a compatibility score.

The role of the first agent, the *Describer* (Table 2), is to define a consistent semantic reference frame for both objects. Given the text labels of $\mathcal{M}_o$ and $\mathcal{M}_b$ (and optionally rendered images), the agent constructs an **object-centric canonical coordinate frame** for each object. This frame specifies (i) an origin on the object (e.g., a natural landmark such as the center of the head or the midpoint between lenses), and (ii) three semantic axes: a rightward axis $(+X)$, an upward axis $(+Y)$, and a forward-facing axis $(+Z)$.

Using these axes, the Describer defines the canonical views of each object—namely which parts of the object correspond to the *front*, *back*, *left*, *right*, *top*, and *bottom*. For example, for glasses the front view corresponds to the side where the lenses face outward, while the back view corresponds to the side facing the wearer. In addition to these axis definitions, the Describer also produces textual cues that help distinguish between ambiguous viewpoints (e.g., differences between left and right sides or between front and back). These semantic rules allow subsequent agents to interpret rendered images of the object in a consistent coordinate system regardless of the camera viewpoint.

In parallel, a second agent, the *Material Verifier* (Table

4), uses the material description generated by the Describer to infer approximate physical parameters of the target object, specifically Young's modulus and Poisson's ratio.

The second agent, the *Scorer* (Table 3), samples several candidate orientations by randomly rotating each object, and aligns $\mathcal{M}_o$ with $\mathcal{M}_b$ by deducing their relative alignment from the candidate orientation that VLM deems to find most consistent (in other words, from whatever candidate orientation that VLM gets least confused by). Given each candidate orientation, we render 4 uniformly-sampled views of both objects. For each rendered image, the scorer predicts two simple properties. First, it determines which side of the object (more precisely, which canonical reference frame) is facing the camera,

$$\texttt{visible_face} \in \{\texttt{front}, \texttt{back}, \texttt{left}, \texttt{right}, \texttt{top}, \texttt{bottom}, \texttt{other}\}.$$

Second, it determines how the object's upward direction appears in the image,

$$\texttt{projected_up} \in \{\texttt{up}, \texttt{down}, \texttt{left}, \texttt{right}, \texttt{other}\}.$$

Intuitively, the first label describes which side of the object (or which side of the canonical frame) the camera sees, while the second label indicates whether the object appears upright, upside down, or rotated sideways in the image.

Before evaluating candidate orientations of $\mathcal{M}_o$, we first establish a consistent reference orientation for $\mathcal{M}_b$. We do this by sampling several uniform azimuthal rotations of $\mathcal{M}_b$, and selecting the rotation that produces the most self-consistent classifications across views.

Once the base orientation is fixed, we generate a small set of discrete orientation hypotheses that account for possible viewpoint ambiguities. Intuitively, this step handles cases where the rendered views correspond to the same physical orientation but appear in a different order due to camera sampling.

For example, suppose the four rendered views of $\mathcal{M}_b$ correspond to the sequence `[front, left, back, right]`. If the base object is rotated by 90° around its vertical axis, the same viewpoints would instead appear as `[left, back, right, front]`. We therefore consider all such cyclic shifts of the rendered view sequence to account for these azimuthal offsets. To handle this case where some objects appear upside down in the image, we also include a hypothesis that applies a 180° in-plane rotation, which reverses the predicted up direction of the object. In total, there are 8 such possible cyclic shifts that account for the mismatch in orientation, all of which we consider as our hypothesis.

For each candidate orientation of $\mathcal{M}_o$, we render M views and classify them using the scorer. The scorer predicts the labels `visible_face` and `projected_up` for each rendered image together with confidence values for each prediction.

To evaluate a candidate orientation, we compare these predicted labels against those of $\mathcal{M}_b$ under each of the discrete orientation hypotheses described above. For every view, we check whether the predicted side of the object (e.g., front, left) and the predicted upward direction match those of the base object. A match contributes positively to the score, and this contribution is weighted by the confidence of the corresponding prediction.

The contributions from all views are then averaged to produce a score for the current hypothesis. In addition, we apply a penalty when the sequence of predicted viewpoints across the rendered views is geometrically inconsistent (for example, if the sequence viewpoints do not follow a plausible progression around the object). The final score for a candidate orientation of $\mathcal{M}_o$ is then the maximum score obtained across all orientation hypotheses.

Intuitively, this scoring procedure favors orientations where the target object consistently appears aligned with the base object across multiple viewpoints. Orientations that produce mismatched sides, inconsistent upward directions, or implausible viewpoint sequences receive lower scores. Now among $N = 20$ orientation of the $\mathcal{M}_o$, we simply select the orientation that achieves the highest score as the final initialization.

The final, initialized target mesh is then obtained by applying a similarity transform to the original target vertices, using the orientation selected as giving the highest score:

$$\mathbf{v}_i^0 = s^* \, R^* \big(\mathbf{v}_i - \bar{\mathbf{v}}\big) + \mathbf{t}^*,$$

where $\mathbf{v}_i$ is a vertex of the original target mesh, $\bar{\mathbf{v}} = \frac{1}{N}\sum_i \mathbf{v}_i$ is the target-mesh centroid, R^* is the final rotation, s^* is the scale, and $\mathbf{t}^*$ is the translation. In our implementation,

$$R^* = (R_b^*)^\top R_h^* R_t^*,$$

where R_t^* is the best target rotation found during VLM search, R_b^* is the rotation initially applied to the base mesh to find the best canonical rotation, and R_h^* is the best hypothesis correction (the cyclic shift we applied, if theres any). The translation is set to

$$\mathbf{t}^* = \mathbf{c}_{\text{region}} + \boldsymbol{\eta},$$

where $\mathbf{c}_{\text{region}}$ is the centroid of the user-defined highlighted region and $\boldsymbol{\eta}$ is an optional Gaussian jitter term. The scale is computed as

$$s^* = 2.0 \cdot \frac{\left\| \mathbf{b}_{\text{region}}^{\max} - \mathbf{b}_{\text{region}}^{\min} \right\|_2}{\left\| \mathbf{b}_{\text{target}}^{\max} - \mathbf{b}_{\text{target}}^{\min} \right\|_2 + \varepsilon}.$$

Material Regularization. In this section, we provide more details of how we formulate the material regularization loss. Recall that we use a hyperelastic material model to compute a `Neo-Hookean` loss function that models the internal forces resisting deformation from each jacobians, J_i. The function takes the deformation gradient F (which we discretely approximate using our jacobians) for each element, along with two material properties: Young's modulus E and Poisson's ratio ν (predicted from our *Material Verifier* agent). These are first converted into Lamé parameters: the shear modulus μ and Lamé's first parameter λ.

$$\mu = \frac{E}{2(1+\nu)} \quad \text{and} \quad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}$$

The function then computes the first Piola-Kirchhoff stress tensor P, which represents the internal forces for each element:

$$P(F) = \mu(F - F^{-T}) + \lambda \ln(\det(F))F^{-T}$$

where $\det(F)$ is the volumetric determinant of the deformation gradient. To apply this continuous model to our discrete mesh, we use our piece-wise Jacobians, J_i to serve as our replacement for the deformation gradient F, so the stress for each face can be approximated as,

$$P(J_i) = \mu(J_i - J_i^{-T}) + \lambda \ln(\det(J_i))J_i^{-T}$$

and subsequently,

$$\mathcal{L}_e = \sum_i P(J_i).$$

We defer more details to [4].

As a result of this deformation, we can obtain a deformed object mesh that tightly and semantically fits around the user-defined region on the base mesh, while strictly abiding by the physically-imposed requirement of non-intersection. We also note that the E value under 1000 makes the $\mathcal{M}_o$ fit too elastically over the $\mathcal{M}_b$, up to a point where it sacrifices semantic realism. We therefore clamp the minimum value of E to 1000 for the purpose of our experiments, which, with an iterative deformation process, successfully models even the most elastic materials we show in our experiments (e.g. "extremely soft cloth").

B. Additional Experiments and Results

B.1. Robustness to Initialization

As noted in the main paper, our method is relatively robust to different initial configurations of the accessory and the base mesh; however, the most adversarial of them can cause it to converge to undesirable local minima. In Figure 2, we run the full optimization from a range of initial orientations and offsets relative to the target region. The method recovers the desired composition for most moderate initializations, and only the most adversarial cases (leftmost and rightmost) fail to converge. These cases motivate the VLM initialization scheme of subsection A.3, which selects a semantically consistent starting configuration and avoids them in practice.

B.2. Additional Qualitative Results

In Figure 3, the same accessory is automatically placed on a wide range of characters, demonstrating how our method can adapt to different scales, head shapes, and body proportions.

B.3. User Study

We conduct an extensive user study for 32 participants, using the same 10 set of examples we used for our quantitative evaluation. For each example, we show 6 randomly ordered results from ours and other baseline methods. We then ask the participants to choose the single best result among the 6 options that "performs the task of putting on a wearable 3D object in the highest quality possible–in a way that best matches the text description provided above in each section." As can be seen from Table 1, our method has consistently been ranked by most users to give the most realistic composition result compared to the other 5 baselines. While our method has an overall average preference of 92.81, it also has an extremely strong minimum preference score (the preference score of the example where the method achieved its lowest user preference among all evaluation examples) of 81.25, meaning that even for the "worst" example, 81.25 percent of users preferred our results over all the baselines.

B.4. Experiment Details

Quantitative Comparison In this section, we will detail how we conduct our quantitative experiments. Among the 5 baselines we compare against in both our quantitative evaluation and user study, we first discuss how we use Instan3dit. Instant3dit takes as input a source mesh and a 3D mask that denotes the local region to make edits. As their 3D mask is not identical to our 2D surface highlights, we manually generate 3D masks that are as close as possible to our 2D highlight. We also note that these 3D masks are more expressive user inputs than our surface highlights, and thus shoudn't be considered a factor that confers our method a quantitative/qualitative edge. One major caveat of comparing against Instant3dit is that the backbone Large Reconstruction Model (LRM) the authors used for their paper is not publicly available (the Adobe LRM). We therefore resorted to the alternative InstantMesh [5] backbone suggested in their GitHub repositories to run the experiments, which apparently degrades the quality of the results.

For quantitative evaluation, we ran Instant3dit from the same text prompt processed by MeshOn's VLM verifiers and the diffusion model used for deformation guidance. We rendered meshes generated from each method in a uniform,

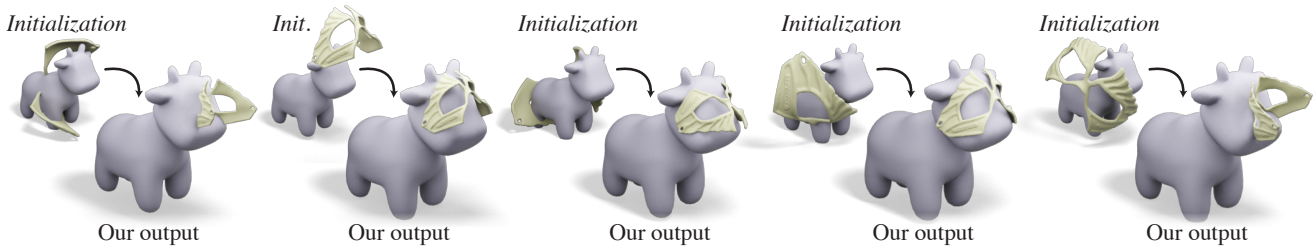


Figure 2. Our method is moderately robust to different initialization configurations, although it can fail to converge to the desired output in very adversarial cases (leftmost and rightmost). In subsection A.3, we introduce a VLM initialization strategy that avoids these cases.

User Study Results						
Statistic	Instant3Dit	Best ICP	Best FGR	RANSAC+ICP	RANSAC+FGR	Ours
Average Preference (%) ↑	2.81	3.13	0.63	0.00	0.63	**92.81**
Minimum (%)	0.00	0.00	0.00	0.00	0.00	81.25
Maximum (%)	15.63	12.50	3.13	0.00	3.13	**100.00**

Table 1. User study results comparing six methods for the 3D wearable compositing task. Each entry reports the percentage of participant selections received by each method across all evaluation prompts (32 participants per prompt). We report the average preference percentage, as well as the minimum and maximum percentage achieved by each method on any individual example.

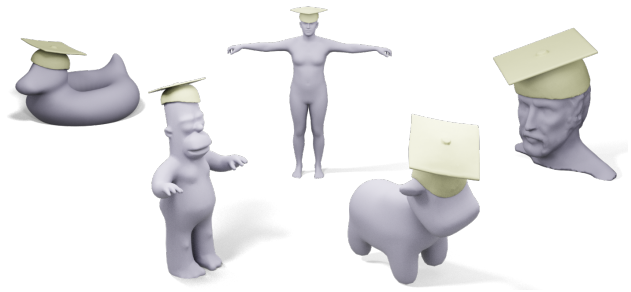


Figure 3. Our method fits the same accessory on different base meshes, adapting to differences in scale, head shape, and body proportions to place the asset in a semantically correct position.

grey textured color from 4 manually sampled viewpoints, and averaged the scores across these viewpoints.

Comparison with Point Registration Methods We also made qualitative/quantitative comparisons against the Point Cloud Registration Methods, including Iterative Closest Points (ICP), Fast Global Registration (FGR), and Random Sample Consensus (Ransac). We note that all of these methods in a standalone setting is not robust enough (or has too much variance) to meaningfully compare against our method. We therefore take standard practices known to improve these methods' performances, and compare the improved results against our method. For ICP, whose results are known to be greatly influenced by initialization, we sample results from $n = 50$ different initializations, and choose the one that gives best RMSE score (we call these **Best ICP** and **Best FGR** since we choose the result from best initialiization). Another approach is to use Ransac (which is relatively better at finding coarse global registrations) as an initialization, and using ICP as a second-step, finetuning method (which we call **Ransac+ICP** and **Ransac+FGR** in our main paper). We compare our results to both methods in the paper. We also found that for this application, an abnormally high threshold value works best in the most stable, robust manner, so we use $th = 5000$ as our ICP threshold.

For FGR, we take the same approach and run comparisons on both multi-step sampling and Ransac initialization method. We use a voxel size of 0.05 and a distance multiplier of 1.5, which is a default value proposed in its `Open3d` implementation.

Examples of user study and quantitative comparison In Figure (4), we provide examples of the renderings we used for conducting user study and quantitative evaluations.

C. Computational Details

We ran all of our experiments on single a40 nvidia gpus. While the runtime depends heavily on the mesh resolution (and the size of user input mask), our method approximately takes around 15-30 minutes to run. The VLM verification stage alone takes a minimum of 5 minutes to run, causing a major bottleneck to the model. While these performance estimates are that of the results we presented in this paper, the total runtime could be greatly reduced without sacrificing too much quality by running each step for significantly less

iterations. We will release the exact parameters and code upon acceptance.

GPU-Optimized BVH memory and runtime test In Figure (1), we provide an extensive experiment of varying the `leaf_size` parameter of our gpu-optimized BVH structure in calculating the vertex-to-triangle distance measure $\hat{d}(v_i, \mathcal{M}_b)$, as well as doing a single fully-batched computation. Specifically, we calculate $\hat{d}(v_i, \mathcal{M}_b)$ using bvh with varying `leaf_size`, applied onto progressively subdivided mesh resolutions. For $\mathcal{M}_o$, we choose a reasonable resolution that is the approximately the average of object meshes we used in our experiments, with 8646 vertices and 17288 faces. We show that `leaf_size=1000` provides a reasonable trade-off between runtime and GPU memory, while running the fully-batched computation without using our BVH structure crashes on high-resolution meshes. We note that the fully-batched computation crashes sooner for lower resolution triangles when employed in our actual optimization pipeline, where factors like autograd trees and backpropagation further constrain gpu memory.

D. Conclusion

We introduced a mesh composition algorithm that produces a meaningful, tight, non-intersecting fit between two shapes, inspired by the application of 3D digital art. For our work to be useful in an interactive, real-time 3D modeling session, however, its runtime would need to improve significantly. We believe there are several potential avenues for this: for example, one could dramatically improve runtimes by building increasingly fine nested simulation cages using existing methods [3]. Then, one could perform the optimizations in our algorithm successively for each refinement level, thus reducing the need for extensive computation on the finest of levels.

As a tool of artist control, our algorithm requires a user to specify a highlighted region of the base object onto which the accessory should be attached. Experimentally, we find that our algorithm's output is sensitive to small changes in this region (see **??**), which could lead to user frustration. This issue could be resolved by future work combining our algorithm with existing text-based mesh selection tools like *3D Highlighter* [1].

As a final step in our fit, our algorithm allows the object to deform in order to better match the base mesh. This results in an improved fit (see **??**) and provides an additional avenue for artist control (see **??**). However, our method still assumes mostly-rigid scenarios in which the attachment to the base mesh is the main source of deformation. For fundamentally elastic materials in which other internal and external forces are dominant (*e.g.*, a piece of cloth draping over a character), a better fit can likely be achieved with existing physical simulation tools.

This is a time of friction between rapidly improving 3D AI research and artist communities reluctant to incorporate these tools due to loss of control and creativity. While solving this friction is a complex problem beyond the scope of any one paper, we hope that non-generative AI-driven tools like MeshOn, which is intended to fit directly into real digital artists' workflows while maximizing creative control, can aid in building bridges between these two worlds.

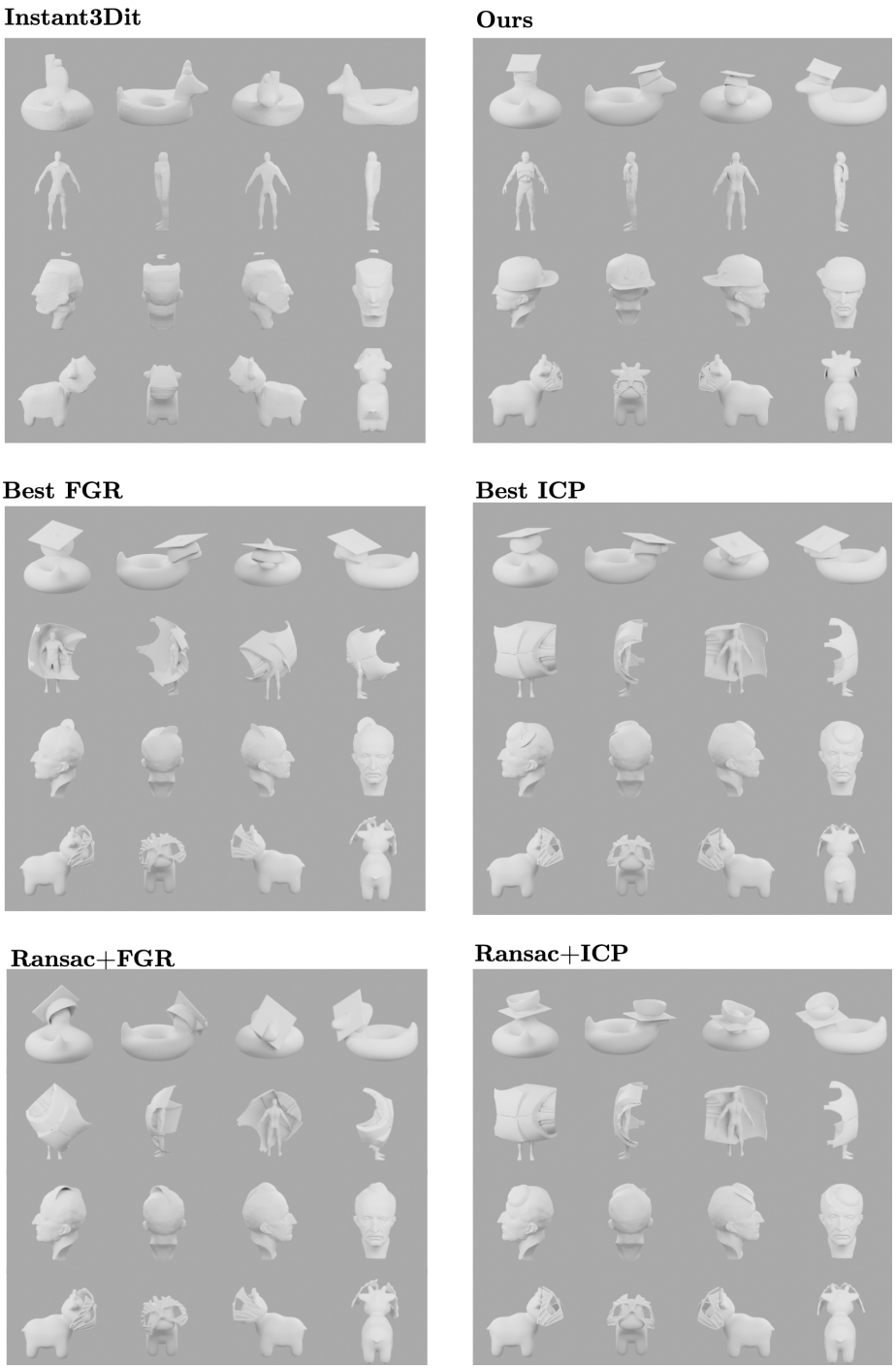


Figure 4. We show examples of renderings that we used for our user study. We also used the same set of examples for evaluating CLIP, CLIP-IQA, and VQA scores, but with a different renderer (nvdiffrast) for technical purposes. We render results from all methods from four viewpoints with greyscale textures.

E. VLM Prompt Templates

Table 2. **Prompt for Agent 1: Semantic Describer.** This agent generates an immutable object-centric canonical reference frame, concise view definitions for both objects, and a material description for the target object.

System Instruction

You are an expert in 3D object semantics, pose, and materials. You MUST define an immutable object-centric canonical frame that will be reused later for view classification. This frame must be stable across renders and must not depend on a specific camera.
OBJECTS:
- **TARGET:** `<target_label>`
- **BASE:** `<base_label>`
- **VISUALS:** [Optional rendered images of target and base meshes from multiple turntable views]
PART 1: OBJECT-CENTRIC CANONICAL FRAMES
For each object, define an object-centric canonical frame, including:
- an origin landmark,
- semantic $+X$ (right),
- semantic $+Y$ (up),
- semantic $+Z$ (front / outward-facing direction),
- at least three cues for distinguishing left vs. right,
- at least three cues for distinguishing front vs. back,
- at least two cues for distinguishing top vs. bottom.

IMPORTANT: The target frame should be defined relative to how the target is functionally worn on the base, but the axes themselves must remain object-centric and reusable across views.
PART 2: REQUIRED CLASSIFIER LABELS
A later classifier will see images of only one object at a time. For each image it must output:
- `visible_face` $\in$ {`front`, `back`, `left`, `right`, `top`, `bottom`, `other`}, indicating the dominant canonical side facing the camera;
- `projected_up` $\in$ {`up`, `down`, `left`, `right`, `ambiguous`}, indicating the image-plane direction of the object's canonical $+Y$ axis.

Provide concise definitions for these labels for both the target and the base.
PART 3: MATERIAL DESCRIPTION
Describe the physical material properties of the **TARGET object only**. This natural-language description will later be converted into approximate simulation parameters.
OUTPUT FORMAT:
You MUST provide your response only as a single valid JSON object with four keys:
`canonical_reference`, `target_view_definitions`, `base_view_definitions`, and `material_description`.

Table 3. **Prompt for Agent 2: View Classifier (Scorer).** This agent classifies views of a *single* object at a time using the immutable object-centric canonical frame generated by Agent 1.

System Instruction

You are a strict viewpoint classifier. You will be shown images of only one object at a time, either the TARGET or the BASE. You must use the immutable object-centric canonical frame below and must not redefine axes for each image.
INPUT CONTEXT:
- **OBJECT ROLE:** `<TARGET or BASE>`
- **OBJECT LABEL:** `<object_label>`
- **OBJECT-CENTRIC CANONICAL FRAME:** `<canonical_reference (from Agent 1)>`
- **CONCISE DEFINITIONS FOR THIS OBJECT:** `<target/base view definitions (from Agent 1)>`
- **VISUALS:** [Rendered images of the object from multiple viewpoints]

TASK:
For each image, output:
- `visible_face` $\in$ {`front`, `back`, `left`, `right`, `top`, `bottom`, `other`}, indicating the dominant canonical side facing the camera;
- `projected_up` $\in$ {`up`, `down`, `left`, `right`, `ambiguous`}, indicating the image-plane direction of the object's canonical $+Y$ axis;
- `face_confidence` $\in [0, 1]$;
- `up_confidence` $\in [0, 1]$;
- a short textual reason citing one or two concrete visual cues.

IMPORTANT RULES:
- Do not mention any other object.
- Do not try to force agreement with any external reference.
- Use only the supplied canonical frame and visual evidence.
- Output only JSON.

OUTPUT FORMAT:
Respond only with a single JSON object containing a list named `view_analysis`. Each entry should contain: `view_pair`, `visible_face`, `projected_up`, `face_confidence`, `up_confidence`, and `reason`.

Table 4. **Prompt for Agent 3: Material Verifier.** This agent converts the natural-language material description produced by Agent 1 into approximate elastic parameters for simulation.

System Instruction
You are a materials expert. Based on the following description of an object, infer approximate elastic parameters for simulation. **INPUT:** Infer simulation parameters from this description of a '<target_label>': '<material_description (from Agent 1)>'. **OUTPUT FORMAT:** Output strictly in JSON with keys: {E, nu, mu}, corresponding to Young's modulus, Poisson's ratio, and shear modulus. **GUIDELINES:** - Units for E and mu are arbitrary but should be proportional across materials. - nu should lie in $[0, 0.5]$. - For technical stability, E must be at least 1000; if the inferred value is smaller, clamp it to 1000.

References

[1] Dale Decatur, Itai Lang, and Rana Hanocka. 3d highlighter: Localizing regions on 3d shapes via text descriptions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20930–20939, 2023. 6

[2] Minchen Li, Zachary Ferguson, Teseo Schneider, Timothy R Langlois, Denis Zorin, Daniele Panozzo, Chenfanfu Jiang, and Danny M Kaufman. Incremental potential contact: intersection- and inversion-free, large-deformation dynamics. *ACM Trans. Graph.*, 39(4):49, 2020. 1

[3] Leonardo Sacht, Etienne Vouga, and Alec Jacobson. Nested cages. *ACM Transactions on Graphics (TOG)*, 34(6):1–14, 2015. 6

[4] Eftychios Sifakis and Jernej Barbic. Fem simulation of 3d deformable solids: a practitioner's guide to theory, discretization and model reduction. In *Acm siggraph 2012 courses*, pages 1–50. ACM, 2012. 4

[5] Jiale Xu, Weihao Cheng, Yiming Gao, Xintao Wang, Shenghua Gao, and Ying Shan. Instantmesh: Efficient 3d mesh generation from a single image with sparse-view large reconstruction models, 2024. 4